

Flap flap flap!

Prepared by: Rene Andre Bedonia Jocsing

Published: February 03, 2026

Deadline: February 27, 2026

Your first machine problem is to create Yet Another Flappy Bird Clone (YAFBC) from scratch in Godot. You may use free-to-use art and sound assets, but the implementation and code must come from you.

Learning Objectives

By accomplishing this machine problem, you should be able to:

- Navigate the Godot engine interface and understand its node-based architecture
 - Implement basic 2D game mechanics including physics, collision detection, and input handling
 - Apply finite state machines to manage game state transitions
 - Work with sprites, animations, and audio integration
 - Implement simple procedural generation systems
 - Demonstrate professional architecture using design patterns
 - Apply fundamental game design principles to create a playable experience
-

Why Flappy Bird?

Flappy Bird is deceptively simple. On the surface, it's just a bird that flaps when you tap the screen, avoiding pipes that scroll endlessly. But underneath that simplicity lies a surprisingly complete gamedev challenge that forces you to grapple with the fundamentals: physics, collision detection, procedural generation, game state management, and the elusive concept of "feel".

More importantly, Flappy Bird is small enough to finish in a reasonable timeframe while being complex enough to matter. You're not just moving a sprite around the screen—you're building a complete game loop, from the moment the player launches the game to the moment they inevitably crash into a pipe and see their score.

This is the sweet spot for our first machine problem. It's large enough that you can't brute-force your way through without understanding what you're doing, but small enough that you won't get lost in the weeds of a sprawling codebase. By the time you're done, you'll have touched every major system that modern games rely on, even if in miniature form.

The Original Mechanics

The Bird

Your bird (or whatever flying object you choose) must respond to player input with a single mechanic: flapping. Each flap applies an upward impulse, fighting against gravity. The bird continuously falls due to gravity, and the player must time their flaps to navigate through a series of obstacles.

The physics here matter more than you think. If gravity is too strong, the game becomes frustrating. Too weak, and it becomes trivial. The same goes for flap strength. Finding the right balance between these values is your first lesson in game feel—the intangible quality that makes a game satisfying to play.

For aesthetics, your bird should also rotate slightly based on its vertical velocity. When falling, it tilts downward. When flapping upward, it tilts up. This small detail provides crucial visual feedback to the player about their current trajectory. Visual feedback is critical for reducing cognitive load in high-pressure physics-based games.

The Pipes

Pipes (or whatever obstacles you choose) spawn at regular intervals and scroll across the screen from right to left. Each pipe pair has a gap that the bird must fly through. The vertical position of this gap should vary randomly within reasonable bounds—you don't want it spawning at the very top or bottom of the screen where it's impossible to reach.

As pipes scroll off the left side of the screen, they should be removed from the scene to avoid accumulating objects in memory. This is called object pooling in more sophisticated implementations, but for now, simply destroying off-screen pipes and spawning new ones is sufficient. Advanced students may implement object pooling for additional credit.

Collision with a pipe ends the game immediately. The hitboxes for collision detection should be fair—players should never feel cheated by a collision that didn't visually make sense. You may introduce other types of obstacles (moving pipes, floating bombs, etc.) as part of your mechanical twist.

Scoring

The player scores one point for each pipe pair they successfully pass through. A simple implementation: when the bird's x-coordinate crosses the center of a pipe's gap, increment the score. However you implement it, ensure you only award the point once per pipe pair.

Display the current score prominently during gameplay. When the player dies, the game should reset cleanly without memory leaks or unreferenced objects. The game shouldn't lag no matter how long it runs.

Game States

Your game needs at least three distinct states managed via a finite state machine:

- **Menu/Idle State:** The initial state before gameplay begins
- **Playing State:** Active gameplay with input response and physics
- **Game Over State:** Triggered on collision, displays score and restart option

Proper state management prevents bugs like the bird continuing to respond to input after dying, or the score incrementing while dead. Review your FSM implementation from Coin Dash.

Your Original Mechanics

What Makes a Valid Twist?

Your clone must include a **unique mechanical twist (or several)** that meaningfully changes gameplay. This is NOT cosmetic.

Valid Examples:

- Gravity reverses on each tap
- Power-ups temporarily change pipe behavior or bird abilities
- Multiplayer racing mode (split-screen or multiple birds?)
- Momentum-based physics (bird accelerates/decelerates)
- Dynamic difficulty scaling based on performance

Invalid Examples:

- Different sprites or visual theme only
- Background music or sound effect changes
- UI/menu styling variations

Ask yourself: Does this change how I play the game, or just how it looks or sounds? Will I get struck for copyright infringement if I publish this on Steam or on Google Playstore?

You may deviate from the Technical Requirements section if and only if: it benefits or is crucial to your original mechanics, or is architecturally sound with justification!

Technical Requirements

Architecture

- **Strong typing:** All function signatures and class members must be type-annotated (whether explicitly through Kotlin-style typehints or implicitly through Python-style walrus operators)
- **State machine:** Use enum-based FSM for game states (idle/playing/gameover)
- **Scene composition:** Separate scenes for Player, Pipe, UI, Main, etc.
- **Signal-driven:** Use signals for decoupled communication (score updates, state changes, etc.)
- **Separation of concerns:** Movement, collision, scoring in distinct methods, and nodes composited into distinct scenes

Common Pitfalls

Spaghetti code will cap your grade at 3.00 regardless of gameplay quality. Architecture is heavily weighted in grading. Code that works but lacks proper organization, design patterns, or architectural integrity will receive a maximum grade of 3.00 for this deliverable.

Input Handling

The game must respond to at least one input method (keyboard, mouse click, or touch). The input should be simple. When pressed, apply some movement(s) to the bird.

Don't allow input buffering or repeated movements from holding the button down. Each press should correspond to exactly one flap. This is crucial for the game's difficulty curve.

Collision Detection

Implement collision detection between:

- The bird and pipes
- The bird and the ground
- The bird and the ceiling

Use Area2D nodes with collision shapes. Ensure collision shapes match visual sprites closely enough that collisions feel fair to the player.

Procedural Generation

Pipes should spawn automatically at regular intervals. Each pipe pair should have a randomly positioned gap. Consider these parameters:

- Spawn interval (how often new pipes appear)
- Pipe gap size (distance between top and bottom pipes)
- Gap position variance (how much the gap's vertical position can vary)
- Pipe scroll speed

These values significantly affect difficulty. Experiment to find settings that create a challenging but fair experience.

Visual and Audio Feedback

At minimum, your game should include:

- Sprites for the bird, pipes, background, and ground
- A flapping animation for the bird (at least 2-3 frames)
- Sound effects for flapping, scoring, and dying
- Background music for different game states

Free assets available at: [OpenGameArt.org](https://opengameart.org/), itch.io, [Kenney.nl](https://kenney.nl/). Ensure assets have appropriate licenses for educational use. Credit all assets in CREDITS.md.

Deliverables

1. The Game

An original playable Godot 4 project based on Flappy Bird (this is your prompt), implementing all core mechanics and technical requirements. The game should be stable—no crashes, no game-breaking bugs. Minor polish issues are acceptable, but the core loop must work flawlessly.

2. *GitHub Repository*

Your complete Godot project must be hosted in a GitHub repository with clean commit history. Include:

- All Godot project files and scenes
- All assets with proper attribution in CREDITS.txt
- README.md containing:
 - Game description and mechanical twist explanation
 - Controls documentation
 - Known issues or limitations
 - Asset credits and licenses
- Built game binary zipped in GitHub Releases section

Invite the instructor as a collaborator. Use semantic commit messages following the conventions from the syllabus.

3. *Presentation (15 minutes)*

Prepare a jam-style presentation covering:

- Live gameplay demonstration
- Architecture decisions and design patterns used
- Explanation of your mechanical twist
- Challenges faced and solutions
- Q&A from classmates and instructor

Class will vote on: Best Architecture, Best Twist, Best Polish. Winners receive +5% extra credit per category. This can stack up to +15%!

Weekly Progress Reports

Weekly progress reports during consultation hours are **strongly encouraged**. Groups that attend receive:

- Ongoing feedback on architecture
- Early detection of design issues
- Code review and refactoring guidance
- Up to +5% extra credit for consistent attendance

Groups that skip progress reports accept the risk of discovering major flaws during final presentations when fixes are costly or impossible, as well as forfeit the extra credit.

Grading Philosophy

This is not a competition to create the fanciest version of Flappy Bird (though you're welcome to be as fancy as you want). The goal is to demonstrate understanding of fundamental game systems, game design, and professional software architecture.

A simple but well-architected game will score better than an ambitious but poorly architected one.

Focus on:

- Clean, maintainable code with proper design patterns
- Correct implementation of core mechanics
- Meaningful mechanical twist
- Professional commit history and documentation
- Excellent game "feel"

Extra features (particle effects, elaborate menus, etc.) are welcome after the foundation is solid, but will not compensate for poor architecture.

Pair Programming Expectations

- Both members must contribute to code and design
- Use Git branches, pull requests, code review
- Maintain clear communication and regular check-ins
- Document design notes in README.md
- Both members must speak during presentation

Individual accountability assessed through:

- GitHub commit history
- Presentation participation
- Consultation session involvement
- Peer evaluation (if issues arise)

See the syllabus for more details regarding the pair programming format.

Academic Honesty

The usage of Large Language Models (e.g., ChatGPT, Claude, Deepseek) to generate code is considered cheating. As cheating is against the university's code of ethics, it is subject to failure in the course and harsh disciplinary action.

You are expected to write your own code and understand every line you submit. During presentations and consultations, you must be able to explain and defend your architectural decisions and implementation choices.

Important Dates

Weekly Progress Reports

Progress reports are conducted during office hours on a first-come, first-served basis. Attendance is optional but **strongly encouraged**, and can earn up to +5% extra credit for the machine problem.

- **Week 1:** February 6, 2026
- **Week 2:** February 10, 2026
- **Week 3:** February 17, 2026
- **Week 4:** February 24, 2026

Submission of GitHub Repository

The GitHub repository for the machine problem must be submitted (with completed code and release build) on **February 27, 2026**. To submit, simply invite the instructor as a collaborator. Late submissions will be penalized with a -25% deduction. Commits dated after the deadline will not be considered during evaluation.

Final Presentation

Machine problem presentations will be conducted in a jam-style format during regular class hours. Both team members must be present and participate.

- **Presentation Date:** Tuesday, March 3, 2026
- **Duration:** 15 minutes per team (demo + Q&A)
- **Format:** Live gameplay demo, architecture showcase and explanation, voting

Note: Machine problems that are functional but exhibit significant architectural issues (e.g., spaghetti code, no FSM, hard-coded values, tight coupling) will receive a maximum grade of 3.00 (60/100) regardless of gameplay quality or presentation performance. Professional game development prioritizes maintainable, scalable systems over short-term functionality.

Grading Rubric

**Score may vary individually for component.*

Criteria	Excellent (90-100%)	Good (75-89%)	Fair (60-74%)	Poor (0-59%)
Architecture & Design (25%)	Clean separation of concerns, proper FSM implementation, signal-driven architecture, demonstrates sound design patterns	Minor architectural issues, most patterns implemented correctly, consistent structure	Significant architectural problems, inconsistent pattern usage, unclear structure	Spaghetti code, no clear architecture, violates OOP principles

Criteria	Excellent (90-100%)	Good (75-89%)	Fair (60-74%)	Poor (0-59%)
Mechanical Twist (20%)	Creative, meaningful gameplay innovation, seamlessly integrated into core loop	Valid twist that changes gameplay meaningfully, decent integration	Minimal twist or poorly integrated, superficial changes	Cosmetic only, missing, or breaks core mechanics
Code Quality (10%)	Strongly typed signatures and members, excellent naming conventions, well-documented using Godot conventions	Mostly typed, good naming, adequate documentation	Inconsistent typing/naming, sparse documentation	Poor typing, unclear code, missing documentation
Individual Contributions* (20%)	Equitable commits with semantic messages, clear authorship, professional Git workflow (branches, PRs)	Mostly balanced contributions, adequate commit messages, basic Git usage	Uneven contributions visible in history, poor commit messages	One member dominates commits, nonsemantic messages, commit noise
Presentation* (25%)	Professional demo with smooth gameplay, clear architecture explanation, both members articulate design decisions, handles Q&A confidently and pitches game well	Good demo, adequate explanations of implementation, both members participate meaningfully, pitch is good	Rough demo with bugs, unclear explanations, uneven participation, struggles with questions, pitch is questionable	Unprepared, can't explain code or architecture, one member dominates, uncertain responses, pitch needs revision

**Score may vary individually for component.*

***Remember: “Code tells you how, comments tell you why.”** — Jeff Atwood, co-founder of Stack Overflow and Discourse